Introduction to Problem Solving and Programming

“Password Strength checker using Rule Based evaluation”

Name : Krissh Raj Rana

Reg No : 25BEC10001

Slot : A14 + D11 + D12

Faculty Name : Mrs Pavithra R

Introduction

in today’s digital era, cybersecurity is becoming increasingly important as users rely on online platforms for communication, banking, and data storage.

one of the most common security vulnerabilities is the use of weak passwords. this project demonstrates how basic programming concepts and string analysis can be used to evaluate password strength.

The system evaluates passwords using various criteria, including length and the presence of uppercase and lowercase letters, numerical digits, and special characters.

the goal is to help users understand how secure their passwords are and encourage them to create stronger passwords.

Problem Statement

the goal is to develop a system that can assess the strength of a password based on already known security rules.

Suppose we give an input password to the system, the system should:

1. analyze its composition

2. assign a strength score

3. classify it as weak, medium, or strong

Functional Requirements

The system must:

1. Accept password input from the user

2. Check password length

3. Detect uppercase and lowercase characters

4. Detect numeric digits

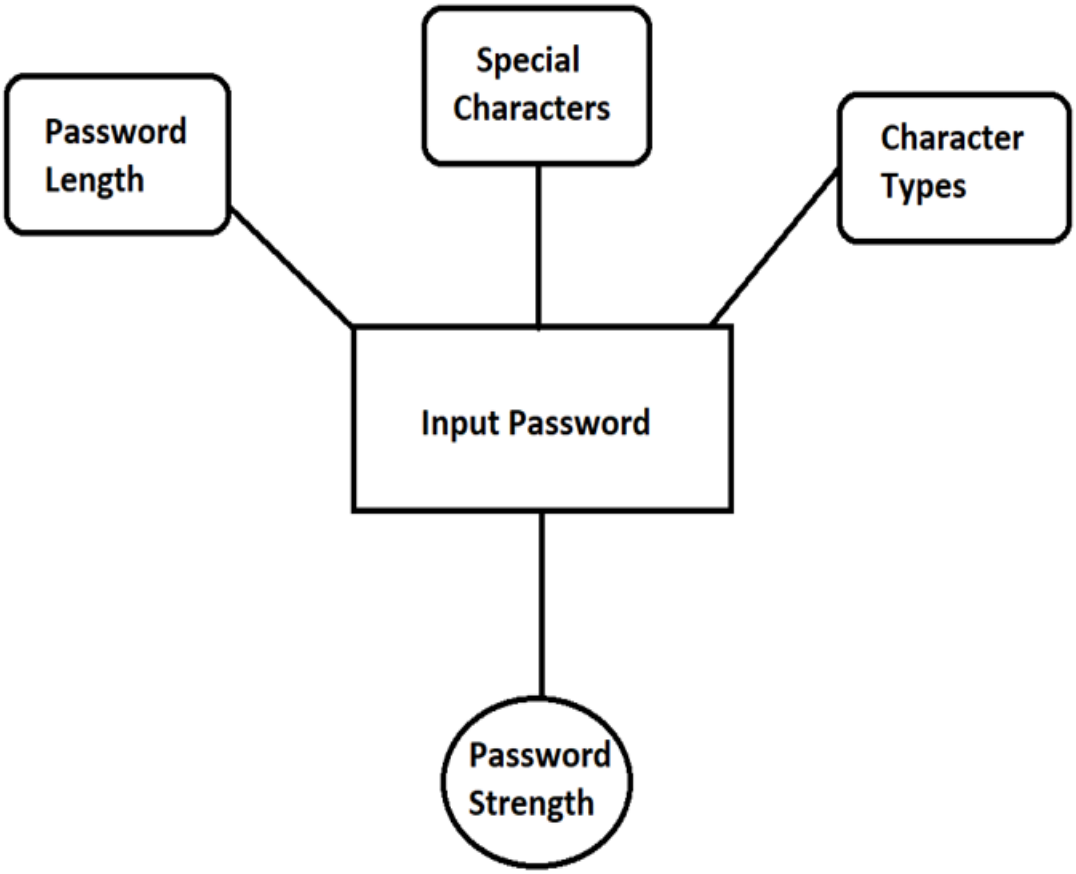
5. Detect special characters

6. Calculate a strength score

7. Display password strength (Weak/Medium/Strong)

8. Provide suggestions for improvement

Conceptual Diagram



Design Decisions & Rationale

1. Rule-based approach chosen due to:

    - Simplicity
    - Easy implementation
    - No need for large datasets

2. Scoring system used to quantify strength

3. Classification (Weak/Medium/Strong) for easy understanding

4. Modular design for future enhancements

Implementation Details

- programming language: python
- libraries used: re (regular expressions)
- algorithm: rule-based evaluation

➢ **major steps:**

1. take password input
2. check length
3. check character types
4. assign score
5. classify strength
6. display result

Python Code

```
import re  #Regular expression module/library


#creating a function to assess the strength of a given password.
def check_password_strength(password):
    score = 0


    if len(password) >= 8: #checking the length of the password.
        score += 1


    if re.search(r"[A-Z]", password): #checking if the password has any uppercase alphabets.
        score += 1


    if re.search(r"[a-z]", password):#checking if the password has any lowercase alphabets.
        score += 1


    if re.search(r"[0-9]", password): #checking if the password has any numbers in it.
        score += 1


    if re.search(r"[!@#$%^&*(),.?\":{}|<>]", password): #checking if the password has any
special characters in it.
        score += 1


        print("Your password score is",score)
    if score <= 2:
        return "It is a Weak Password"
    elif score <= 4:
        return "It is a Medium Password"
    else:
```

return "It is a Strong Password"

print("This program assess your input password's strength\n")

print("If the password score <=2 , it is a weak password")

print("If the password score <=4 , it is a medium password")

print("If the password score > 4 , it is strong password")

A=input("\nEnter your password : ")#Entering your password

print(check_password_strength(A))

- **Code in python** :-

```
import re  #Regular expression module/library

#creating a function to assess the strength of a given password.
def check_password_strength(password):
    score = 0

    if len(password) >= 8: #checking the length of the password.
        score += 1

    if re.search(r"[A-Z]", password): #checking if the password has any uppercase alphabets.
        score += 1

    if re.search(r"[a-z]", password):#checking if the password has any lowercase alphabets.
        score += 1

    if re.search(r"[0-9]", password): #checking if the password has any numbers in it.
        score += 1

    if re.search(r"[!@#$%^&*(),.?\":{}|<>]", password): #checking if the password has any special characters in it.
        score += 1

        print("Your password score is",score)
    if score <= 2:
        return "It is a Weak Password"
    elif score <= 4:
        return "It is a Medium Password"
    else:
        return "It is a Strong Password"


print("This program assess your input password's strength\n")
print("If the password score <=2 , it is a weak password")
print("If the password score <=4 , it is a medium password")
print("If the password score > 4 , it is strong password")

A=input("\nEnter your password : ")#Entering your password
print(check_password_strength(A))
```

Screenshots and outputs

- **Output for a weak Password : -**

```
This program assess your input password's strength

If the password score <=2 , it is a weak password
If the password score <=4 , it is a medium password
If the password score > 4 , it is strong password

Enter your password : abc333
It is a Weak Password
```

- **Output for a Medium Password : -**

```
This program assess your input password's strength

If the password score <=2 , it is a weak password
If the password score <=4 , it is a medium password
If the password score > 4 , it is strong password

Enter your password : Agbc221
It is a Medium Password
```

- **Output for a Strong Password : -**

```
This program assess your input password's strength

If the password score <=2 , it is a weak password
If the password score <=4 , it is a medium password
If the password score > 4 , it is strong password

Enter your password : AdhfHH@1449
Your password score is 5
It is a Strong Password
```

Testing Approach

- Unit testing for password function
- Testing with different password types
- Edge case testing (empty, short passwords)

Test Cases :-

- **1st Case :-**

Lowercase letters + numbers

- 2nd Case :-

Lowercase and Uppercase letters + numbers

- 3rd Case :-

Lowercase and Uppercase letters + numbers + special characters

⬍ Challenges Faced :-

- Defining correct strength rules
- Handling edge cases
- Making scoring fair and meaningful

References

- **Python official documentation**

- **Regular Expressions (re module)**

- **Cybersecurity best practices**